



TÉCNICO
LISBOA

Autonomous Maze Solving with MDPs and Model-Free RL

Gonçalo Silva 102995
Pedro Serradas 100065
Rodrigo Trindade 107224
Salvador Santos 100367

Autonomous Systems

Project Statement

- Develop and implement algorithms to **solve mazes** using **MDPs** and **model-free RL methods**.
- Test and analyse implementations using **simulation** and **real-world** environments.
- The target robot is the **AlphaBot2 with camera**.
- System be built on top of **ROS2** in **python**.

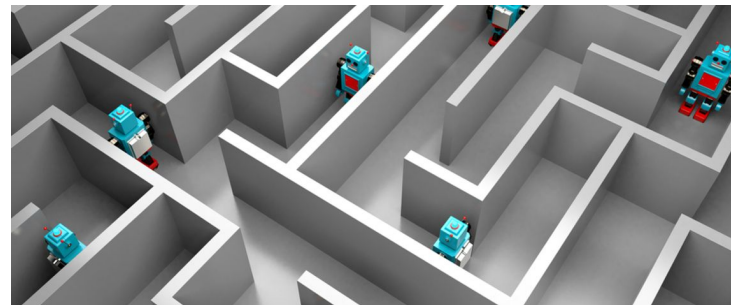


Main Preparation Readings

- **Course Materials** (Lab Guide, Project presentation, etc).
- **Reinforcement Learning: An Introduction (chaps. 3 and 6)**: Richard S. Sutton and Andrew G. Barto 2018 MIT Press.
- **Artificial Intelligence: A Modern Approach (chaps. 7, 8, and 10)**: Stuart Russell and Peter Norvig 2009 Pearson.

Decision Making

- Actions have **uncertain outcomes** → require **stochastic** decision-making.
- Framework: **Markov Decision Process (MDP)**.
- Key assumption: **Markov Property** - next state depends only on current state and action.



MDP Formulation

S	Set of all possible states	e.g. discrete grid cells; topological graphs.
A	Set of available actions	e.g. move forward; turn 90° left.
$p(s', r s, a)$	Transition probability	e.g. goes forward with $p=0.8$; slips right with $p=0.1$.
$R(s, a)$	Reward function	e.g. +100 goal reached; -1 per step; -10 hitting a wall
$\gamma \in [0,1]$	Discount factor	e.g. $\gamma=0$ - only next reward matters; $\gamma=1$ - look into future rewards.

- **Agent acts sequentially:** $S_0, A_0, R_1, S_1, A_1, R_2, S_2, A_2, R_3, \dots$
- **Goal:** find a **policy** $\pi(a | s)$ that maximizes **expected cumulative (discounted) reward**.

$$G_t \doteq \sum_{k=t+1}^T \gamma^{k-t-1} R_k,$$

MDP Formulation

State-value function: expected return when starting in s and following π .

$$v_{\pi}(s) \doteq \mathbb{E}_{\pi}[G_t \mid S_t=s] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t=s\right], \text{ for all } s \in \mathcal{S},$$

Action-value function: expected return starting from s , taking the action a .

$$q_{\pi}(s, a) \doteq \mathbb{E}_{\pi}[G_t \mid S_t=s, A_t=a] = \mathbb{E}_{\pi}\left[\sum_{k=0}^{\infty} \gamma^k R_{t+k+1} \mid S_t=s, A_t=a\right]$$

Bellman Optimality Equation:

$$v_{*}(s) = \max_{a \in \mathcal{A}(s)} q_{\pi_{*}}(s, a) = \max_a \sum_{s', r} p(s', r \mid s, a) [r + \gamma v_{*}(s')]$$

Solving the MDP: Value Iteration

- One of the simplest to solve the Bellman Optimality Equation and obtain the optimal policy is **Value Iteration**.
- This is a **dynamic programming** approach that repeatedly updates $V(s)$ until convergence.

Value Iteration, for estimating $\pi \approx \pi_*$

Algorithm parameter: a small threshold $\theta > 0$ determining accuracy of estimation

Initialize $V(s)$, for all $s \in \mathcal{S}^+$, arbitrarily except that $V(\text{terminal}) = 0$

Loop:

| $\Delta \leftarrow 0$

| Loop for each $s \in \mathcal{S}$:

| $v \leftarrow V(s)$

| $V(s) \leftarrow \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$

| $\Delta \leftarrow \max(\Delta, |v - V(s)|)$

until $\Delta < \theta$

Output a deterministic policy, $\pi \approx \pi_*$, such that

$$\pi(s) = \arg \max_a \sum_{s',r} p(s',r|s,a) [r + \gamma V(s')]$$

Solving the MDP: Model-Free

Value Iteration finds the optimal policy, but it requires:

- Full knowledge of the transition model $p(s', r \mid s, a) \rightarrow$ **model-based**
- Complete state enumeration $\rightarrow$ **all states must be known in advance**
- Computational cost scales as $O(|S|^2 \cdot |A|)$ per sweep $\rightarrow$ **poor scalability**

This motivates **Model-Free Reinforcement Learning**:

- Learn optimal behaviour directly from experience, **without ever knowing the environment model.**

Temporal Difference (TD) Learning

TD-learning borrows the best of two worlds:

- Monte Carlo: Learn from **actual sampled returns**.
- Dynamic Programming: Update from **current value estimates**.

The TD error at each step:

$$\delta_t \doteq R_{t+1} + \gamma V(S_{t+1}) - V(S_t).$$

- Small $\delta \rightarrow$ current estimate is accurate.
- Large $\delta \rightarrow$ significant update needed.

SARSA vs Q-Learning

Two main TD control algorithms arise:

	SARSA	Q-Learning
Type	On-policy	Off-policy
Update uses	Actual next action a'	Greedy best action $\max Q(s', a')$
Update rule	$Q(S_t, A_t) + \alpha [R_{t+1} + \gamma Q(S_{t+1}, A_{t+1}) - Q(S_t, A_t)]$	$Q(S_t, A_t) + \alpha [R_{t+1} + \gamma \max_a Q(S_{t+1}, a) - Q(S_t, A_t)]$
Learns	Value of the policy currently followed	Optimal Q^* regardless of exploration policy
Behaviour	Conservative: avoids being close to risky areas (e.g. walls).	Aggressive: targets optimal path directly and greedily.

Practical insight: SARSA is safer during training; Q-Learning usually converges faster and better to a final policy.

Workplan

1. Problem statement, literature readings, workplan.
2. Design and explain the algorithms to implement, including choosing state and action space representations.
3. Micro-simulator: implement MDP + Value Iteration, SARSA and Q-Learning, and validate on synthetic maze.
4. ROS2 real-robot integration and testing in physical maze.
5. Systematic experiments: multiple runs, varying mazes; SARSA vs Q-Learning vs Value Iteration comparison.

Open Questions & Conclusions

Some Open Questions

- How to best **represent states and actions**?
- How to **model environment** for value iteration?
- **Number of training episodes** needed for convergence in a physical maze?
- **Best reward shaping** for efficient maze solving?

Conclusions

- **MDPs** provide a **stochastic framework for decision-making** under uncertainty.
- Value Iteration is the simpler, baseline method but **model-dependent** and **computationally limited**.
- TD methods overcome this: they **learn purely from experience**.
- **Clear workplan**: theory → micro-simulator → real robot → comparative analysis.